Arquitetura escolhida: camadas (Apresentação, Aplicação, Domínio e Infraestrutura), com dependências apenas no sentido descendente.

Por que camadas?

- **Separação de responsabilidades:** cada camada tem um motivo para mudar. A home dinâmica e a busca mudam na apresentação. As regras de recomendação e o fluxo dos casos de uso (UC01–UC06) mudam na aplicação. Os conceitos de negócio ficam no domínio, e o banco e o envio de push ficam na infraestrutura.
- **Coerência com o que já foi projetado:** o `CtrlHome` do diagrama de sequência (GRASP *Controller*) fica no pacote de controladores. O `motorRecomendacao` (*Pure Fabrication*) tem pacote próprio, para que a lógica de recomendação não contamine as classes de domínio. O modelo de domínio (Usuário, Conteúdo, Gênero, Sessão) fica isolado no pacote de domínio.
- **Baixo acoplamento e alta coesão:** a interface do usuário e a do administrador compartilham os mesmos controladores. Assim, a regra de negócio não é duplicada entre as duas.
- **Substituição de tecnologias:** o domínio define interfaces (por exemplo, repositório e notificador), e a infraestrutura as implementa. Dá para trocar o SGBD ou o provedor de push sem alterar o domínio nem a aplicação.
- **Testabilidade:** domínio e aplicação podem ser testados sem banco de dados e sem interface.

Por que não as outras opções?

- **Microsserviços:** o sistema tem escopo e equipe pequenos. O custo de comunicação entre serviços, implantação e consistência de dados não se justifica.
- **Arquitetura limpa/hexagonal:** é a mais próxima do que fizemos, mas exige mais cerimônia (casos de uso como classes, portas e adaptadores explícitos). Para o porte do projeto, camadas entregam o mesmo benefício principal, que é isolar o domínio.

Regra de dependência: cada camada só conhece a imediatamente inferior. Nenhuma camada inferior conhece as superiores.